CYBERSECURITY AVANZATA · LEZIONE 3 · PASSWORD, HASH E SALT

Il login che non si fa rubare

Scrivi in Java il cuore di un sistema di login: registrazione, verifica e un limite ai tentativi. Se domani qualcuno ruba il tuo "database", deve trovarci solo impronte lente e salate.

Cosa consegni: un file **login-cognome.zip** con **Login.java** e un **README.pdf** di una pagina, su Google Classroom. **Scadenza:** quella indicata su Novaula.

Che cosa devi fare

1. Crea una classe `Utente` con nome, salt e hash, e una `Map<String, Utente>` come database.
2. `registra(nome, char[] pw)`: rifiuta password più corte di **12 caratteri**, genera 16 byte di salt con `SecureRandom`, calcola **PBKDF2WithHmacSHA256** con 600.000 iterazioni e salva.
3. `accedi(nome, char[] pw)`: ricalcola e confronta con `MessageDigest.isEqual`. Dopo **5 errori** di fila blocca l'utente per 60 secondi.
4. Bonus (+1): prima di registrare, controlla la password con Pwned Passwords (k-anonimato, come in laboratorio) e rifiutala se compare.
5. Scrivi un `main` che dimostra: registrazione, accesso giusto, accesso sbagliato, blocco dopo 5 errori. Nel README incolla l'output e spiega in 5 righe le tue scelte.

Esempio di output

`registra anna ... ok (312 ms)` · `accedi anna [giusta] ... true` · `accedi anna [sbagliata] x5 ... bloccato per 60 s`

> *"Non c'è dubbio che, col tempo, le persone si affideranno sempre meno alle password."* · **Bill Gates**, *2004. Nel frattempo, conserviamole bene.*

Come viene valutato

Criterio	Punti
Salt casuale per utente e PBKDF2 con i parametri richiesti	3
Verifica corretta con confronto a tempo costante	2
Limite ai tentativi funzionante	2
Nessuna password in chiaro salvata o stampata	1
Demo e README chiari	2

Usa solo password di prova inventate per il compito, mai le tue.